Problem 1

(a)

Assume that inverter, AND, and OR gates have 1ns delay and XOR, XNOR gates have 3ns delay. What is the worst case delay from any input to any sum output for the 4-bit carry lookahead adder shown in the figure below. What is the worst case delay from any input to the carry output? The structure of the CLA(figure 1) is as discussed in class. The figures below should help to guide you.

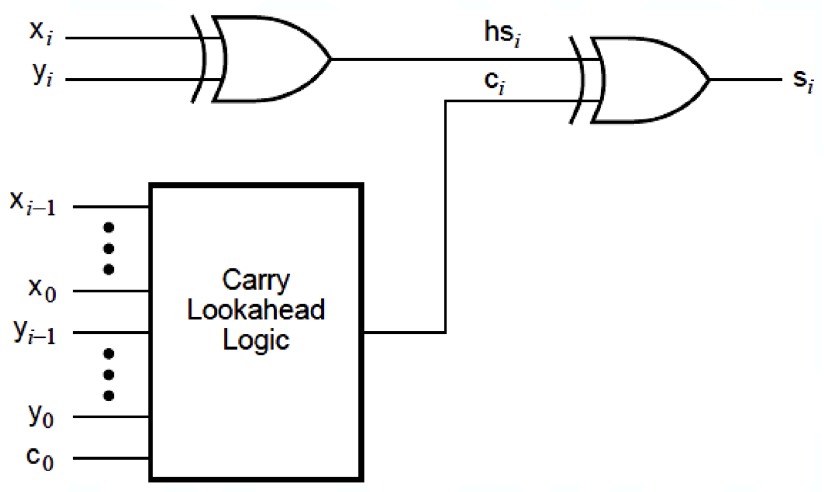

Figure 1: Structure of one stage of a carry lookahead adder

$$c_1 = g_0 + p_0 c_0 = 1 + (3+1) = 5 \text{ ns},$$

$$\begin{aligned} c_2 &= g_1 + p_1 \cdot c_1 \\ &= g_1 + p_1 \cdot (g_0 + p_0 \cdot c_0) \\ &= g_1 + p_1 \cdot g_0 + p_1 \cdot p_0 \cdot c_0 = 1 + max(1, (3+1), (3+1)) = 5 \text{ ns}, \end{aligned}$$

$$\begin{aligned} c_3 &= g_2 + p_2 \cdot c_2 \\ &= g_2 + p_2 \cdot (g_1 + p_1 \cdot g_0 + p_1 \cdot p_0 \cdot c_0) \\ &= g_2 + p_2 \cdot g_1 + p_2 \cdot p_1 \cdot g_0 + p_2 \cdot p_1 \cdot p_0 \cdot c_0 = 1 + (3+1) = 5 \text{ ns}, \end{aligned}$$

$$\begin{aligned} c_4 &= g_3 + p_3 \cdot c_3 \\ &= g_3 + p_3 \cdot (g_2 + p_2 \cdot g_1 + p_2 \cdot p_1 \cdot g_0 + p_2 \cdot p_1 \cdot p_0 \cdot c_0) \\ &= g_3 + p_3 \cdot g_2 + p_3 \cdot p_2 \cdot g_1 + p_3 \cdot p_2 \cdot p_1 \cdot g_0 + p_3 \cdot p_2 \cdot p_1 \cdot p_0 \cdot c_0 = 5 \text{ ns}, \end{aligned}$$

$$s_0 = x_0 \oplus y_0 \oplus c_0 = p_0 \oplus c_0 = 3 \text{ ns},$$

$$\begin{aligned} s_1 &= p_1 \oplus c_1 \\ &= x_1 \oplus y_1 \oplus (g_0 + p_0 \cdot c_0) = 3 + 1 + max(1, (3+1)) = 8 \text{ ns}, \end{aligned}$$

$$s_2 = p_2 \oplus c_2 = x_2 \oplus y_2 \oplus c_2 = 3 + 5 = 8 \text{ ns},$$

$$s_3 = p_3 \oplus c_3 = x_3 \oplus y_3 \oplus c_3 = 3 + 5 = 8 \text{ ns},$$

Answer:

We defined the two notations to form the above relations.

$$\text{Generate signal: } g_i = x_i y_i \tag{1}$$
$$\text{Propagate signal: } p_i = x_i \oplus y_i \tag{2}$$

The worst-case delay from any input to any sum output is 8 ns, and the worst-case delay from any input to the carry output is 5 ns. In this problem, we consider all the gates have unlimited fan-in, for example, $s_o = x_0 \oplus y_0 \oplus c_0$ could be done in one xor operation instead of using two xor gates.

(b)

Determine the worst-case propagation delay of the multiplier in Figure below, assuming that the propagation delay from any full-adder input to its sum output is twice as long as the delay to the carry output. Repeat, assuming the opposite relationship. If you were designing the adder cell from scratch, which path would you favor with the shortest delay?

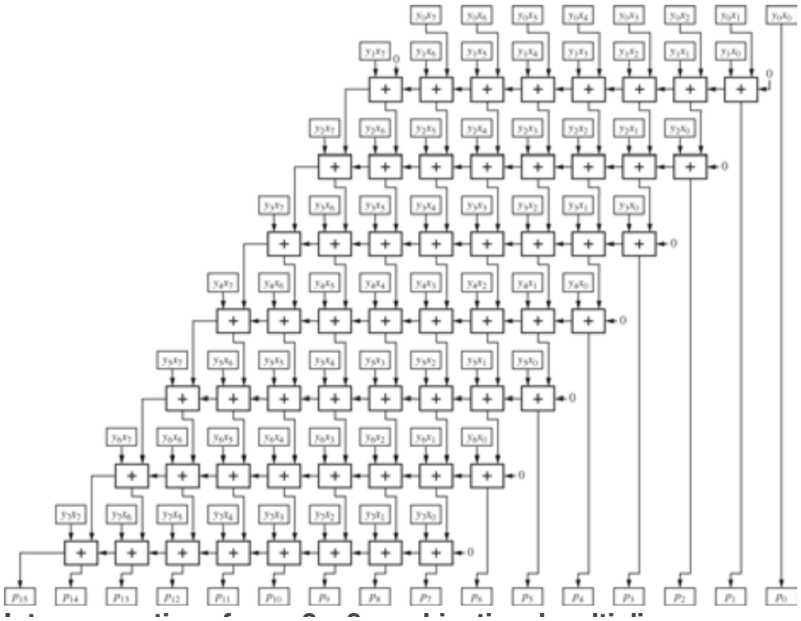

Figure 2: Interconnections for an 8 × 8 combinational multiplier.

Answer:

Let T_S and T_C denote the propagation delays from any input of a full adder to its sum and carry outputs, respectively. The critical signal path passes through roughly 6 sum paths and 13 carry paths, with one additional output transition that can be either a sum or carry. This leads to the following two cases:

a If $T_S = 2T_C = 2T$, the longest delay would be

$$7T_S + 13T_C = 27T_C = 27T$$

b If $T_C = 2T_S = 2T$, the longest delay would be

$$14T_C + 6T_S = 34T_S = 34T$$

Thus, if we are designing the adder cell, we will manage to **design the carry path faster**, as it appears more times in the critical path.

Problem 2

What is the delay through the critical path in the 8-bit Kogge-Stone adder? Assume that it gets implemented with 2-input simple logic gates (and, or, exor, not) and that a gate delay equals 1 unit of time. Trace the path through the adder by listing the number and type of each gate.

Answer:

For each processing component in the Kogge-Stone adder, with the inputs $(g_{\text{in}_1}, p_{\text{in}_1})$ from the current bit and $(g_{\text{in}_2}, p_{\text{in}_2})$ from a preceding bit, the output pair $(g_{\text{out}}, p_{\text{out}})$ is computed as

$$g_{out} = g_{in_1} + p_{in_1} \cdot g_{in_2}$$
$$p_{out} = p_{in_1} \cdot p_{in_2}$$

Note that all the g_{in_i} and p_{in_i} is only related to the current bit x_i and y_i.The longest carry propagation path passes through three processing components (since $\log_2 8 = 3$ levels in an 8-bit adder).Assuming all initial (g_i, p_i) pairs are computed in parallel at the beginning, each processing component introduces a delay of two gate levels (one AND gate followed by one OR gate) due to the computation of g_{out}. Therefore, the total delay for the carry to reach the most significant bit is

$$\text{Delay} = 2 \times 3 + 1 = 7 \text{ units},$$

where the additional one unit corresponds to the initial generation of $g_i = x_i \wedge y_i$. If the sum bit S_7 is also considered, the final XOR operation adds one extra gate delay, resulting in a total of

$$7 + 1 = 8 \text{ units of time.}$$

Problem 3

Write RTL-style Verilog module for a 3×3 (i.e., each input is 3 bits) combinational multiplier and simulate by using the following 16 input combinations.

- Input 1 = 5, Input 2 = all possible value combinations
- Input 1 = all possible value combinations, Input 2 = 2

Use explicit **assign** statements to define each partial product calculation, and you can use **assign** statements with arithmetic addition to accumulate those partial products.

CE303 > HW3 > mul3by3.v

```
1   `timescale 1ns/1ps
2
3   module mul3by3(A, B, Cout);
4       input [2:0] A, B;
5       output [5:0] Cout;
6
7       wire [2:0] p0 = A&{3{B[0]}};
8       wire [2:0] p1 = A&{3{B[1]}};
9       wire [2:0] p2 = A&{3{B[2]}};
10
11      wire [5:0] p0_ext = {3'b000,          p0        };
12      wire [5:0] p1_ext = {2'b00,   p1,     1'b0      };
13      wire [5:0] p2_ext = {1'b0,    p2,     2'b00     };
14
15      assign Cout = p0_ext + p1_ext + p2_ext;
16
17  endmodule
18
```

(a) 3×3 multiplier Verilog RTL module.

CE303 > HW3 > mul3by3_tb.v

```
1   `timescale 1ns/1ps
2   module tb_mul3by3;
3       reg  [2:0] A, B;
4       wire [5:0] Cout;
5
6       mul3by3 dut(.A(A), .B(B), .Cout(Cout));
7
8       task run_case(input [2:0] a, input [2:0] b);
9       begin
10          A = a; B = b; #1;
11          if (Cout !== a*b) begin
12              $display("FAIL  A=%0d B=%0d  C=%0d (expected %0d)", a, b, Cout, a*b);
13          end else begin
14              $display("PASS  A=%0d B=%0d  C=%0d", a, b, Cout);
15          end
16      end
17      endtask
18
19      integer i;
20      initial begin
21          // A = 5, B = 0..7
22          for (i = 0; i < 8; i = i + 1) run_case(3'd5, i[2:0]);
23          // B = 2, A = 0..7
24          for (i = 0; i < 8; i = i + 1) run_case(i[2:0], 3'd2);
25          $finish;
26      end
27  endmodule
28
```

(b) Testbench for multiplier with giving specific input

Figure 3: RTL design for multiplier and its corresponding testbench

```
xrun: 18.09-s011: (c) Copyright 1995-2019 Cadence Design Systems, Inc.
Loading snapshot worklib.tb_mul3by3:v ..................... Done
xcelium> source /vol/cadence2018/XCELIUM1809/tools/xcelium/files/xmsimrc
xcelium> run
PASS  A=5 B=0  C=0
PASS  A=5 B=1  C=5
PASS  A=5 B=2  C=10
PASS  A=5 B=3  C=15
PASS  A=5 B=4  C=20
PASS  A=5 B=5  C=25
PASS  A=5 B=6  C=30
PASS  A=5 B=7  C=35
PASS  A=0 B=2  C=0
PASS  A=1 B=2  C=2
PASS  A=2 B=2  C=4
PASS  A=3 B=2  C=6
PASS  A=4 B=2  C=8
PASS  A=5 B=2  C=10
PASS  A=6 B=2  C=12
PASS  A=7 B=2  C=14
Simulation complete via $finish(1) at time 16 NS + 0
./mul3by3_tb.v:25        $finish;
xcelium> exit
```

Figure 4: Simulation output according to the multiplier with its testbench

Problem 4

The table below lists all shift functions available in a barrel shifter. The Verilog code provides a partial template for a barrel shifter illustrating how one of the shift functions (Vrol) can be implemented in Verilog and embedded into the barrel shifter. Following this example complete the barrel shifter implementation by adding the functions for the remaining shift operations. Simulate all options of the barrel shifter by applying
DIN = [1001011101010011]
S = [0011]

Shift Type	**Name**	**Code**	**Function**	**Note**
Left rotate	Lrotate	000	Vrol	Wrap-around
Right rotate	Rrotate	001	Vror	Wrap-around
Left logical	Llogical	010	Vsll	0 into LSB
Right logical	Rlogical	011	Vslr	0 into MSB
Left arithmetic	Larith	100	Vsla	0 into LSB
Right arithmetic	Rarith	101	Vsra	Replicate MSB

```
module Vrbarrel16(DIN, S, C, DOUT);
    input [15:0] DIN;       // Data inputs
    input [3:0] S;          // Shift amount, 0-15
    input [2:0] C;          // Mode control
    output [15:0] DOUT;     // Data but output
    reg [15:0] DOUT;
    parameter   Lrotate = 3'b000, //Define the coding of
                Rrotate = 3'b001,//the different shift modes
                Llogical = 3'b010,
                Rlogical = 3'b011,
                Larith = 3'b100,
                Rarith = 3'b101;

    function [15:0] Vrol;
        input [15:0] D;
        input [3:0] S;
        integer ii, N;
        reg [15:0] TMPD;
        begin
            N = S; TMPD = D;
            for (ii=1;ii<=N; ii=ii+1) TMPD = {TMPD[14:0], TMPD[15]};
            Vrol = TMPD;
        end
    endfunction

    function [15:0] Vror;
        input [15:0] D;
        input [3:0] S;
        integer ii, N;
        reg [15:0] TMPD;
        begin
            N = S; TMPD = D;
            for (ii=1;ii<=N;ii=ii+1) TMPD = {TMPD[0], TMPD[15:1]};
            Vror = TMPD;
        end
    endfunction
```

(a) Barrel Shifter Verilog Code part 1

```
module Vrbarrel16(DIN, S, C, DOUT);

    function [15:0] Vsll;
        input [15:0] D;
        input [3:0] S;
        integer ii, N;
        reg [15:0] TMPD;
        begin
            N = S; TMPD = D;
            for (ii=1;ii<=N;ii=ii+1) TMPD = {TMPD[14:0],1'd0};
            Vsll = TMPD;
        end
    endfunction

    function [15:0] Vsrl;
        input [15:0] D;
        input [3:0] S;
        integer ii, N;
        reg [15:0] TMPD;
        begin
            N = S; TMPD = D;
            for (ii=1;ii<=N;ii=ii+1) TMPD = {1'd0, TMPD[15:1]};
            Vsrl = TMPD;
        end
    endfunction

    function [15:0] Vala;
        input [15:0] D;
        input [3:0] S;
        integer ii, N;
        reg [15:0] TMPD;
        begin
            N = S; TMPD = D;
            for (ii=1;ii<=N;ii=ii+1) TMPD = {TMPD[14:0],1'd0};
            Vala = TMPD;
        end
    endfunction
```

(b) Barrel Shifter Verilog Code part 2

```
module Vrbarrel16(DIN, S, C, DOUT);

    function [15:0] Vara;
        input [15:0] D;
        input [3:0] S;
        integer ii, N;
        reg [15:0] TMPD;
        begin
            N = S; TMPD = D;
            for (ii=1;ii<=N;ii=ii+1) TMPD = {TMPD[15], TMPD[15:1]};
            Vara = TMPD;
        end
    endfunction


    always @ (DIN or S or C)
        case(C)
            Lrotate : DOUT = Vrol(DIN,S);
            Rrotate : DOUT = Vror(DIN,S);
            Llogical: DOUT = Vsll(DIN,S);
            Rlogical: DOUT = Vsrl(DIN,S);
            Larith : DOUT = Vala(DIN,S);
            Rarith : DOUT = Vara(DIN,S);
            default: DOUT = DIN;
        endcase
endmodule
```

(c) Barrel Shifter Verilog Code part 3

```
`timescale 1ns/1ps
module tb_shifter;
    reg [15:0] Din;
    reg [3:0] S;
    reg [2:0] C;
    wire [15:0] Dout;

    Vrbarrel16 dut(.DIN(Din), .S(S), .C(C), .DOUT(Dout));

    task run_case(input [15:0] d, input [3:0] s, input [2:0] c);
    begin
        Din = d;
        S = s;
        C = c;
        #1;
        $display("Case : [%3b] Output : Dout=%16b --> %0d",
            c, Dout, Dout);
    end
    endtask


    integer C_case;
    integer Giv_DIN = 16'b1001011101010011;
    integer Giv_S = 4'b0011;
    // integer Giv_S = 4'b0001;
    initial begin

        $display("Original: Din=%0b --> %0d", Giv_DIN, Giv_DIN);
        $display("-----------------------------------");
        for (C_case=0; C_case<=5; C_case=C_case+1) run_case(
            Giv_DIN,Giv_S,C_case[3:0]
            );
        $finish;
    end
endmodule
```

(d) Testbench for shifter with giving specific input

Figure 5: RTL design for Barrel Shifter and corresponding testbench

```
xcelium> run
Original: Din=1001011101010011 --> 38739
-----------------------------------
Case : [000] Output : Dout=1011101010011100 --> 47772
Case : [001] Output : Dout=0111001011101010 --> 29418
Case : [010] Output : Dout=1011101010011000 --> 47768
Case : [011] Output : Dout=0001001011101010 --> 4842
Case : [100] Output : Dout=1011101010011000 --> 47768
Case : [101] Output : Dout=1111001011101010 --> 62186
Simulation complete via $finish(1) at time 6 NS + 0
./shifter_tb.v:32         $finish;
xcelium> exit
```

Figure 6: Simulation output according to the multiplier with its testbench under given input of Din, S, and six shift types